

MySQL 中的分组聚合与排序展示

在 MySQL 中，`GROUP BY` 和 `ORDER BY` 是两个完全不同但经常配合使用的子句。它们的核心区别如下：

核心区别总结

特性	<code>GROUP BY</code> (分组)	<code>ORDER BY</code> (排序)
主要目的	将数据按某列 归类汇总	将结果集按某列 排列顺序
处理对象	针对 一组 数据进行聚合计算	针对 每一行 数据进行排序
常用搭配	聚合函数 (<code>SUM</code> , <code>COUNT</code> , <code>AVG</code>)	排序关键字 (<code>ASC</code> , <code>DESC</code>)
结果行数	通常会 减少 行数 (合并同类项)	不改变 行数, 只改变顺序
执行顺序	先执行 (在 <code>WHERE</code> 之后)	后执行 (通常在最后)

1. GROUP BY：分组与聚合

作用：把具有相同值的行归为一组，通常配合聚合函数使用，用于统计分析。

示例场景：统计每个部门有多少员工，或者每个部门的平均工资。

代码块

```
1  -- 统计每个城市 (city) 的平均收入 (Revenue)
```

```
2 SELECT city, AVG(Revenue) as avg_revenue
3 FROM sales_table
4 GROUP BY city;
```

逻辑过程：

1. 把所有“北京”的行找出来，算一个平均值。
 2. 把所有“上海”的行找出来，算一个平均值。
 3. 结果集中，每个城市只会出现一行。
-

2. ORDER BY：排序与展示

作用：对查询出来的结果进行升序或降序排列，不影响数据的聚合逻辑。

示例场景：查看销售记录，并按金额从高到低排列。

代码块

```
1  -- 查询所有销售记录，并按收入降序排列
2  SELECT *
3  FROM sales_table
4  ORDER BY Revenue DESC;
```

逻辑过程：

1. 取出所有数据。
2. 比较每一行的 `Revenue` 字段。
3. 按照从大到小的顺序重新排列输出。

3. 两者配合使用（经典案例）

在实际开发中，我们经常先分组统计，然后再对统计结果进行排序。

示例：找出平均收入最高的前 3 个城市。

代码块

```
1  SELECT city, AVG(Revenue) as avg_revenue
2  FROM sales_table
3  GROUP BY city           -- 第一步：按城市分组，算出每个城市的平均值
4  ORDER BY avg_revenue DESC -- 第二步：按算出的平均值从高到低排序
5  LIMIT 3;               -- 第三步：只取前3名
```

关键注意事项

1. 执行顺序：

SQL 的执行顺序通常是：FROM -> WHERE -> **GROUP BY** -> HAVING -> SELECT -> **ORDER BY** -> LIMIT。

这意味着你必须先分组，才能对分组后的结果进行排序。

2. SELECT 字段的限制：

在使用 **GROUP BY** 时，**SELECT** 后面只能出现两类字段：

- 出现在 **GROUP BY** 后面的字段（如 **city**）。
- 被聚合函数包裹的字段（如 **SUM(Revenue)**）。

如果你尝试直接 `SELECT city, name FROM table GROUP BY city`，在严格模式下会报错，因为 `name` 既没分组也没聚合。

3. 性能影响：

- `GROUP BY` 通常需要进行**哈希运算**或**临时表排序**，开销较大。
- `ORDER BY` 如果涉及大量数据且没有索引，也会触发**文件排序（Filesort）**，影响性能。



总结口诀

`GROUP BY` 是为了“算总数/算平均”，把多行变一行；

`ORDER BY` 是为了“排座次/看先后”，让数据更有序。